Automation Template Selenium Java Cucumber

User Manual

v1.3.0 — Docker & CI Integration
Stable / Validated / Published
Release date: October 12, 2026

Java 21 default / Java 17 supported - Selenium 4.48.0 - Cucumber 7.34.7 - JUnit 5.13.4 - Gradle 8.14.5

Contents

1. First run
2. Architecture
3. Stack, prerequisites and VS Code
3.1 Configure Java 17
4. Configuration and commands
4.1 Using Docker
5. First automation
6. Reuse
7. CI/CD contract and support
8. GitHub Actions workflow
9. Run and inspect CI
10. Quality gate and evidence
11. Main branch protection
12. Logging and observability
12.1 Event flow
12.2 Levels
12.3 Execution and privacy
12.4 Diagnostics
13. Automatic failure evidence
14. Generate and inspect reports
15. Glossary

Purpose

This manual covers setup, configuration, execution, extension, CI, evidence, reporting and diagnosis for the Web UI automation template. The included example uses a local HTML fixture. Basic Java knowledge is needed to create new tests.

Version history

Version	Date / status	Main changes
v1.3.0	2026-10-12 UTC Published	Docker & CI Integration.
v1.2.0	Oct 8, 2026	Stable / Validated / Published. Logging, failure evidence, Cucumber HTML/JSON reporting.
v1.1.0	Oct 1, 2026	Stable / Validated / Published. CI quality gate and test artifacts.
v1.0.2	Sep 26, 2026	Published documentation-only hotfix.
v1.0.1	Sep 25, 2026	Published hardening and CI/CD readiness.
v1.0.0	Sep 18, 2026	First stable template release.

The release date above is GitHub Release published_at in UTC. The original Milestone 4 candidate audit remains a historical record, not the current release status.

1. First run

Install JDK 21 and Chrome or Edge. Open PowerShell at the repository root and run the Gradle Wrapper. Global Gradle is not needed.

```
.\gradlew.bat clean test -Dheadless=true
```

A visible run uses -Dheadless=false and requires a graphical desktop. On Linux/macOS use ./gradlew. Open build/reports/cucumber/cucumber.html after the run. The sample scenario checks the local HTML fixture, so it does not rely on the content of an external website.

2. Architecture

Gradle -> JUnit Platform runner -> Cucumber feature -> Before hook -> Steps -> Page objects -> WebDriver -> After hook -> evidence and reports. Configuration precedence: JVM -D, environment, then config.properties.

Layer	Responsibility
Features	Gherkin scenarios under src/test/resources/features.
Steps	Map Gherkin to page actions and assertions.
Pages	Hold locators, explicit waits and UI interactions.
Driver / Hooks	Create and close a per-thread Chrome or Edge session.
Support	Manage the driver reference and failure evidence.

Keep locators out of steps, scenario assertions out of pages, and business logic out of hooks.

3. Stack, prerequisites and VS Code

Component	Version and use
Java	21 default, 17 supported toolchain.
Gradle Wrapper	8.14.5 build and dependencies.
Selenium Java	4.48.0 browser control.
Cucumber Java/Engine	7.34.7 BDD/Gherkin.
JUnit BOM	5.13.4 JUnit Platform.
SLF4J / Logback	2.0.20 / 1.6.5 test logging.

Check java -version, .\gradlew.bat --version, and git --version. In VS Code open the root with code ., install Java/Gradle and Gherkin support, and wait for Gradle import.

3.1. Configure Java 17

Only 17 and 21 are accepted by -PjavaVersion. Set JAVA_HOME to an installed JDK 17 for the current shell, add its bin directory to PATH, confirm java -version, then run .\gradlew.bat clean test -PjavaVersion=17. In VS Code, select JDK 17 in Java: Configure Java Runtime. Restore JDK 21 and use -PjavaVersion=21 to return to the default. CI uses Java 21.

4. Configuration and commands

config.properties defaults: browser=CHROME, headless=false, baseUrl=https://example.com/, timeoutSeconds=15, screenshotOnFailure=true. Override them with JVM -D properties or BROWSER, HEADLESS, BASE_URL, TIMEOUT_SECONDS and SCREENSHOT_ON_FAILURE environment variables. javaVersion is a Gradle -P property.

Goal	Command
Headless suite	.\gradlew.bat clean test -Dheadless=true
Visible suite	.\gradlew.bat clean test -Dheadless=false
Edge	.\gradlew.bat clean test -Dbrowser=EDGE
Java 17	.\gradlew.bat clean test -PjavaVersion=17
Tag @example	.\gradlew.bat test "-Dcucumber.filter.tags=@example"
Alias	.\gradlew.bat cucumber

The sample uses a local fixture regardless of baseUrl. A derived project can use baseUrl in its own page objects.

4.1. Using Docker

Docker runs the framework in a reproducible Linux container, separate from Windows Java and browser installations. The published v1.3.0 release contains Temurin 21, Chrome and ChromeDriver 155.0.8059.39, Gradle Wrapper 8.14.5, Selenium, Cucumber, JUnit, and POM. Its status is Stable / Validated / Published as of October 12, 2026 UTC (ISO: 2026-10-12).

Download the project from GitHub

Install Git for Windows from git-scm.com/download/win and open PowerShell. Clone the source, enter the directory, and list remote branches:

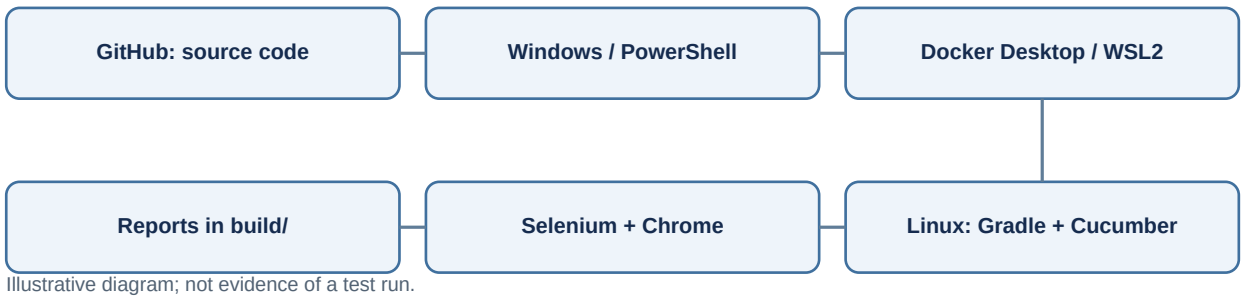
```
git --version
git clone https://github.com/IsraelBaz-coder/selenium-java-cucumber-automation-template.git
Set-Location .\selenium-java-cucumber-automation-template
git fetch origin
git branch -r
```

Select a Docker-enabled ref such as main. Check all three files:

```
Test-Path .\Dockerfile
Test-Path .\gradlew
Test-Path .\build.gradle
git check-attr eol -- gradlew
```

All three Test-Path results must be True and Git must show gradlew: eol: lf. The correction since d173e7d is part of main.

git clone downloads source; docker build creates the image locally; docker pull would require an image published in a registry. No published and verified prebuilt image is available yet.



Illustrative diagram; not evidence of a test run.

Diagram description: GitHub supplies source to Windows; PowerShell asks Docker Desktop/WSL2 to start the container; Gradle and Cucumber run Selenium with Chrome; results are written under build/.

1. Windows, WSL2, and Docker Engine

Goal: Prepare the Linux engine. **Requirements:** Installed Docker Desktop, WSL2, and virtualization.

```
wsl --status
docker version
docker info
```

Explanation: Open Docker Desktop from Start and wait for the running engine; both Client and Server must appear. **Expected result:** Server shows Docker Desktop and Linux.

Recognize an error: No Server or daemon unavailable. **Resolution:** Start or restart Docker Desktop and confirm Linux mode; for WSL trouble run wsl --update and restart if requested.

2. PowerShell in the correct directory

Goal: Use this Dockerfile as the build context. **Requirements:** Repository downloaded to Windows.

```
Get-Location
Test-Path .\Dockerfile
Test-Path .\gradlew
Test-Path .\build.gradle
git check-attr eol -- gradlew
```

Explanation: Work in the cloned directory after selecting a Docker-enabled ref. **Expected result:** All three Test-Path commands return True and eol reports lf.

Recognize an error: False or eol unspecified means a wrong directory or revision without the fix. **Resolution:** Use the correct folder and a revision with gradlew text eol=lf in .gitattributes.

3. Build the image

Goal: Create the development image. **Requirements:** Running engine, correct root, and download access.

```
docker build --progress=plain -t selenium-java-cucumber-template:1.3.0-dev .
```

Explanation: --progress=plain shows each step, -t names and tags the image, and the dot selects the current folder. **Expected result:** Build ends without ERROR and makes the local image available.

Recognize an error: ERROR or nonzero exit code. **Resolution:** Check daemon, network, proxy, and certificates. Do not disable TLS for TLS/PKIX.

4. Verify components

Goal: Inspect image versions. **Requirements:** Built image.

```
docker run --rm selenium-java-cucumber-template:1.3.0-dev java -version
docker run --rm selenium-java-cucumber-template:1.3.0-dev google-chrome --version
docker run --rm selenium-java-cucumber-template:1.3.0-dev chromedriver --version
```

Explanation: --rm removes each temporary container; the final argument replaces the test command. **Expected result:** Java 21; Chrome and ChromeDriver 155.0.8059.39.

Recognize an error: Missing image or different browser and driver versions. **Resolution:** Rebuild from the root and confirm CHROME_VERSION in Dockerfile.

5. Run and interpret

Goal: Run the headless smoke test. **Requirements:** Available image.

```
docker run --rm selenium-java-cucumber-template:1.3.0-dev
```

Explanation: Without extra arguments, ./gradlew --no-daemon test runs. Without a mount, build/ disappears with the container. **Expected result:** WebDriver initialized successfully, Example Domain PASSED, and BUILD SUCCESSFUL.

Recognize an error: BUILD FAILED or a red step; WARN alone does not mean failure. **Resolution:** Read the first real error. TECH-001 (CDP) remains OPEN; TECH-002 (selector) is CLOSED.

6. Retain reports

Goal: Keep build/ on Windows after --rm. **Requirements:** Shared folder and UID 10001 write access.

```
New-Item -ItemType Directory -Force .\build | Out-Null
$out = (Resolve-Path .\build).Path
$bind = "type=bind,source=$out,target=/home/automation/app/build"
docker run --rm --mount $bind selenium-java-cucumber-template:1.3.0-dev
```

Explanation: The bind mount links host and container build/ to retain HTML, JSON, XML, and a log on Windows. **Expected result:** BUILD SUCCESSFUL and files in build/reports/ and build/logs/. No failure PNG was validated because the scenario passed.

Recognize an error: Permission denied or missing reports. **Resolution:** Check folder, Docker Desktop, and UID 10001 permissions; see docs_en/DOCKER.md.

Errors and warnings

If ./gradlew: not found although it exists, check git check-attr eol -- gradlew: it must report lf; chmod +x cannot fix CRLF. Stopped daemon: start Docker Desktop. WSL2: use wsl --status and wsl --update. Downloads: inspect --progress=plain. TLS/PKIX: check certificates and proxy. Permissions: check UID 10001. The CDP warning is TECH-001 OPEN; the Cucumber selector is TECH-002 CLOSED. Chrome needs --no-sandbox in this container; the process runs as UID 10001. CI retains only reports, XML, logs, available screenshots, and container.log; classes and caches are excluded.

The LF rule is part of main. v1.3.0 is the latest official published release. Final validation confirmed BUILD SUCCESSFUL locally with headless=true and headless=false using --offline, a PASSED Cucumber scenario, and Docker 6/6. Local CDP 153 versus 152 and Docker CDP 155 versus 152 warnings appeared; the Cucumber/JUnit selector warning did not. quality-gate and docker-tests passed in the PR and main. The base image is pinned by digest, but apt packages may vary in future uncached builds.

Docker in GitHub Actions

The CI workflow starts on pull requests targeting main and pushes to main. quality-gate keeps Java 21 and Gradle; docker-tests builds the Dockerfile and runs Selenium/Cucumber with headless Chrome. It does not publish the image. A test failure leaves the Docker job in Failure.

On GitHub open Pull requests -> your PR -> Checks -> docker-tests -> Details to read Build test image and Run headless tests and collect evidence. You can also open Actions -> CI -> run and select both jobs. Under Artifacts download docker-test-evidence when present; it includes container.log and generated reports, XML results, logs, and screenshots under build/. Upload is attempted even on failures and files are retained for 14 days. If execution stops before files are generated, the artifact may be absent.

Recorded PR and main validation confirmed quality-gate, docker-tests, and both artifacts. v1.3.0 is published with Stable / Validated / Published status.

5. First automation

Add a .feature under src/test/resources/features, Java step definitions under steps, and a page object under pages. Place stable locators and explicit waits in the page, and scenario assertions in steps. Start with the included example_domain.feature and ExampleDomainPage.

Example feature

```
# language: en
Feature: Login
Scenario: Successful login
Given the user opens the login page
When the user signs in with valid credentials
Then the dashboard is displayed
```

Create LoginPage.java with private locators and intention-revealing open(), login() and isDashboardVisible() methods. Create LoginSteps.java using DriverManager, delegate UI actions to the page and assert the expected outcome with JUnit. Set the target URL through -DbaseUrl. Prefer WebDriverWait over Thread.sleep.

6. Reuse

Replace the sample with the target application, externalize test data, run the tagged scenario, then the full suite. Keep credentials, generated build output, and screenshots out of Git.

For a derived repository, update rootProject.name in settings.gradle and group in build.gradle. Replace sample features, pages and steps only after the smoke test works. Use tags such as @smoke and @regression to select suites. Review the included CI workflow before relying on it for a new application.

7. CI/CD contract and support

.github/workflows/ci.yml runs on pull requests to main and pushes to main. It uses ubuntu-latest, Temurin Java 21, Gradle cache, and ./gradlew clean test -Dheadless=true. A nonzero Gradle exit code fails the job. The docker-tests job runs the image in this workflow; it does not publish images or use new secrets. Selenium Grid and automated deployment are not implemented.

8. GitHub Actions workflow

Workflow: checkout -> Java 21 -> Gradle setup/cache -> headless test -> upload available test-evidence. The artifact includes Cucumber HTML/JSON, Gradle HTML, JUnit XML, logs and eligible failure screenshots; it is retained for 14 days. Upload runs with if: always() and cannot turn FAIL into PASS.

9. Run and inspect CI

For a pull request, open Checks -> quality-gate or docker-tests -> Details. Inspect the failed step and download test-evidence from Actions when available. Fix the branch and rerun. An early compilation failure may leave incomplete artifacts.

Step-by-step PR review

Create a focused branch, edit and test locally, then inspect git status and git diff for secrets and generated files. Commit the change and push the branch. On GitHub select main as the PR base and your branch as compare. Open Checks to see checkout, Java setup, Gradle cache, tests and evidence upload. Every new pushed commit reruns the check. In Actions, open the workflow run and both jobs; read the status text, not only its color.

When quality-gate fails, expand the failed step, keep the full error, download test-evidence if present, fix the branch, rerun .\gradlew.bat clean test -Dheadless=true locally, then commit and push. Wait for the new check. The artifact may contain fewer files if compilation stopped before tests.

10. Quality gate and evidence

The main ruleset documented by the project requires a pull request and passing quality-gate. A passing scenario produces no failure screenshot; an eligible failed scenario can produce a PNG and Cucumber image/png attachment.

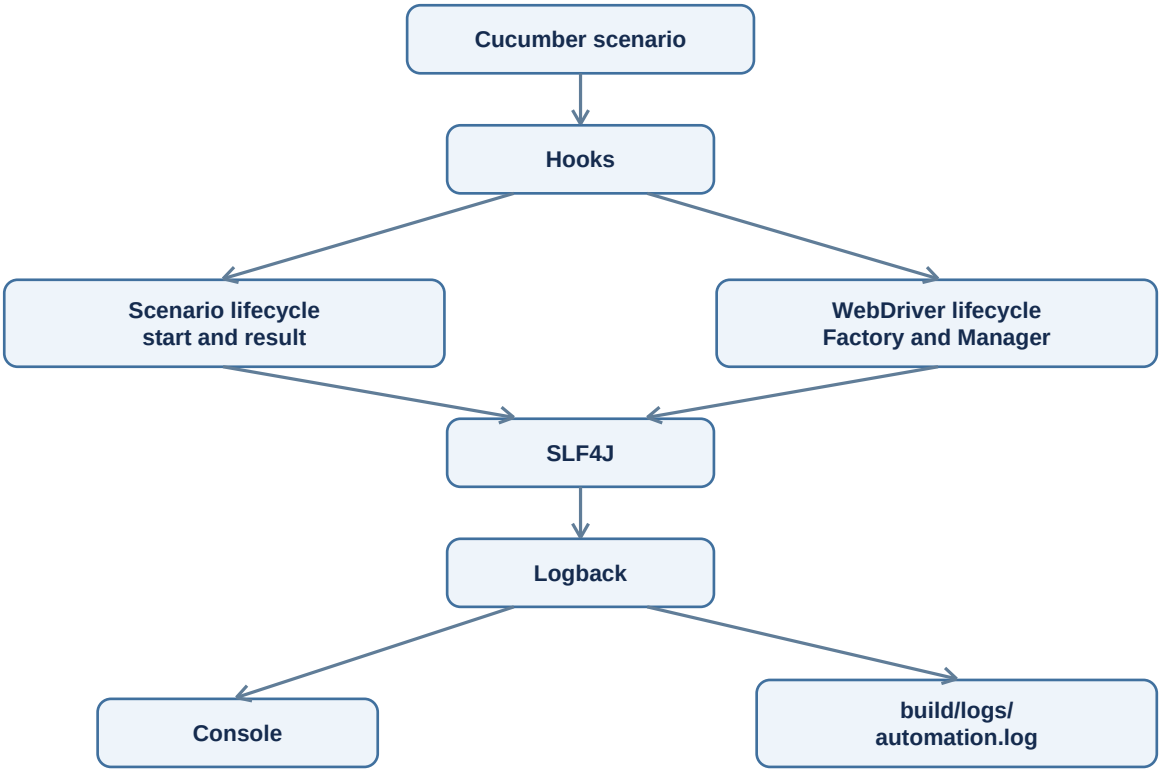
11. Main branch protection

Review and CI validation precede merge. A required check that fails or remains pending blocks integration. Do not move an existing release tag to correct documentation.

12. Logging and observability

SLF4J 2.0.20 in hooks and driver code delegates to Logback 1.6.5, configured by src/test/resources/logback-test.xml. Logback writes console output and build/logs/automation.log.

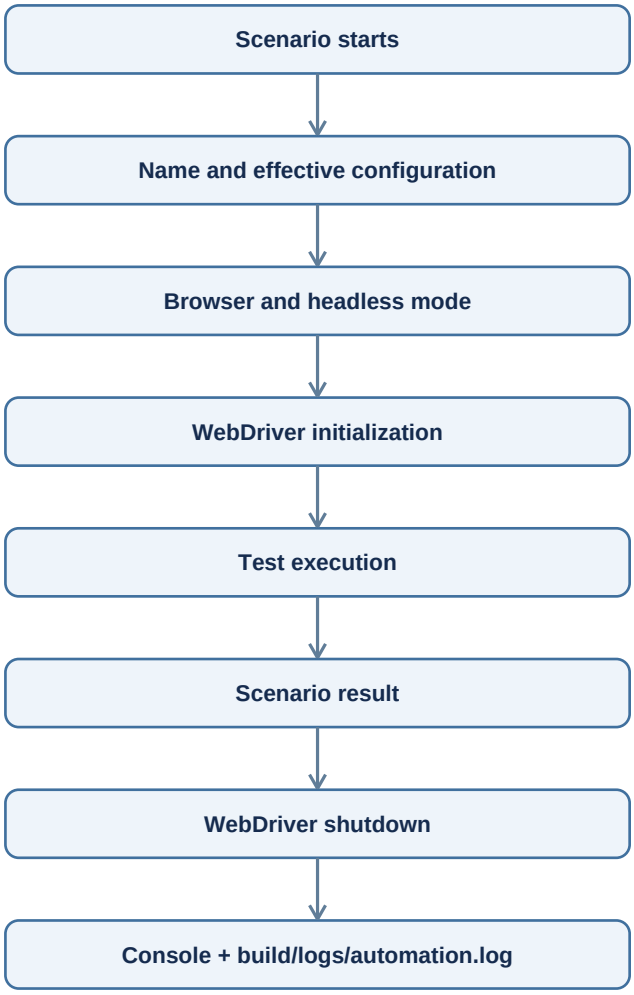
Diagram A - logging architecture



12.1. Event flow

Scenario start and effective browser/headless setting -> WebDriver initialization -> test execution -> scenario result -> optional failure capture -> driver shutdown. Logs record events throughout.

Diagram B - observability flow



12.2. Levels

DEBUG: technical driver detail (hidden by default). INFO: normal scenario and driver lifecycle. WARN: recoverable capture issue. ERROR: evidence or shutdown error. Root level is INFO.

12.3. Execution and privacy

The framework does not log baseUrl, screenshot bytes or Base64. Never add passwords, tokens, cookies or full environment dumps. Third-party exception traces, screenshots and reports can contain application data; review them before sharing.

12.4. Diagnostics

If no log appears, check that test started and that clean did not remove prior output. To inspect dependencies use .\gradlew.bat dependencies --configuration testRuntimeClasspath. For PKIX errors inspect the JDK trust store and proxy; do not disable TLS.

13. Automatic failure evidence

On failed scenarios with screenshotOnFailure=true, the After hook passes WebDriver to EvidenceManager before shutdown. An active screenshot-capable session saves a PNG under build/evidence/screenshots/ and attaches image/png to Cucumber. The filename contains a sanitized scenario name, timestamp and UUID. Capture errors generate warnings without replacing the original test failure.

14. Generate and inspect reports

Output	Path
Cucumber HTML	build/reports/cucumber/cucumber.html
Cucumber JSON	build/reports/cucumber/cucumber.json
Gradle HTML	build/reports/tests/test/
JUnit XML	build/test-results/test/
Logback	build/logs/automation.log
Failure PNG	build/evidence/screenshots/

Match scenario name and status across Cucumber HTML/JSON and the log, then inspect XML/Gradle HTML. clean removes prior results; inspect evidence before rerunning. No historical dashboard or cross-run aggregation is implemented.

To diagnose a failure, open the Cucumber HTML and locate the failed step and image attachment. Compare scenario name and status with automation.log and JUnit XML. If capture was enabled and a driver was active, inspect the PNG. A missing PNG on a PASS is expected. A screenshot capture failure must not hide the original scenario failure. Review reports and images for sensitive application data before sharing.

15. Glossary

Term	Definition
BDD	Behavior-driven development; scenarios describe expected behavior.
Gherkin	Readable language for features, scenarios and steps.
Page Object Model	Pattern that encapsulates UI locators and interactions.
WebDriver	Selenium interface that controls Chrome or Edge.
Hook	Code that prepares or cleans up a scenario.
Gradle Wrapper	Versioned scripts that run the required Gradle version.
Headless	Browser execution without a visible window.
Quality gate	Required check used before integrating a change.
Artifact	Downloadable files produced by a CI run.
Screenshot	PNG image saved as evidence on an eligible failure.
Automation testing	Tests executed by software to verify an application.
Selenium	Library for automating web browsers.
Cucumber	BDD tool that runs Gherkin scenarios.
Feature	Business capability described in a Gherkin file.
Scenario	Concrete example that verifies expected behavior.
Step definition	Java code that implements a Gherkin sentence.
JUnit Platform	Platform that discovers and coordinates tests.
Gradle	Build, dependency and test execution tool.
Locator	Rule that identifies a UI element.
CSS selector	Rule that selects elements by CSS attributes or classes.
XPath	Syntax to locate elements in a DOM.
Assertion	Check of an expected test result.
Test data	Controlled data used by a test run.
Environment variable	External value that configures a run.
Explicit wait	Wait until a WebDriver condition holds.
CI	Continuous integration of changes through automated checks.
CD	Continuous delivery or deployment; this template does not deploy.

Glossary (continued)

Term	Definition
CI/CD	Continuous integration and delivery processes.
Pipeline	Automated sequence of build and validation steps.
Git	Distributed version control system.
GitHub	Repository hosting and CI platform.
GitHub Actions	Service that runs repository workflows.
Workflow	Instructions in a file such as ci.yml.
Job	Group of steps run by a CI runner.
Step	Individual instruction in a CI job.
Runner	Temporary machine executing a CI job.
ubuntu-latest	GitHub Actions Ubuntu runner label.
Pull request	Proposal to integrate branch changes after review.
Branch	Separate line of Git development.
main	Shared primary repository branch.
Feature branch	Branch dedicated to a specific change.
Status check	Visible result of a GitHub verification.
Required check	Check that must pass before merging.
Branch protection	Rules restricting changes to a branch.
Log	Chronological record of execution events.
Report	Readable summary of test results.
Exit code	Process result: zero success; nonzero failure.
PASS	Verification completed successfully.
FAIL	Verification ended with an error.
Commit	Identifiable snapshot of Git changes.
Push	Transfer of local commits to a remote repository.
Merge	Integration of one branch into another.
Checkout	Retrieval of a revision for execution.